

Структура згідно методичних рекомендацій	«Чекліст» компонентів, які обов'язково мають бути продемонстровані (позначені як [_]), а опціональні — як [X] (також усі блок-схеми алгоритмів)	Допоміжні матеріали (назва теки), надані протягом семестру на практичних заняттях
ТИТУЛЬНА СТОРІНКА		
ЗАВДАННЯ НА КУРСОВЕ ПРОЄКТУВАННЯ		
АНОТАЦІЯ		
ЗМІСТ		
ВСТУП		
1. ОГЛЯД МЕТОДІВ ТА СПОСОБІВ ПРОЄКТУВАННЯ ТРАНСЛЯТОРІВ		
2. ФОРМАЛЬНИЙ ОПИС ВХІДНОЇ МОВИ ПРОГРАМУВАННЯ 2.1. Деталізований опис вхідної мови в термінах розширеної нотації Бекуса-Наура 2.2. Опис термінальних символів та ключових слів	EBNF: [_] Код (boost::spirit), що <u>пострічково</u> відповідає EBNF та коректно опрацьовує тестові програми: [_]	EBNFVerify+GrammarVerify EBNFVerify+GrammarVerify verify_syntax_by_EBNF__2025
3. РОЗРОБКА ТРАНСЛЯТОРА З ВХІДНОЇ МОВИ ПРОГРАМУВАННЯ		
3.1. Вибір технології програмування 3.2. Проектування таблиць транслятора та вибір структур даних		

3.3. Розробка лексичного аналізатора 3.3.1. Розробка блок-схеми алгоритму 3.3.2. Опис програми реалізації лексичного аналізатора	Початковий етап: Регулярний вираз для токенизатора: [_]. Регулярний вираз підмови ключових слів: [_]. Регулярний вираз підмови ідентифікаторів: [_]. Регулярний вираз підмови беззнакових літералів із фіксованою комою: [_]. Код на основі регулярних виразів коректно опрацьовує тестові програми: [_]. Безпосередній етап: Детальний опис процесу перетворення регулярних виразів у недетерміновані скінченні автомати (NFA): [X]. Детальний опис процесу видалення λ-переходів для отриманих NFA: [X]. Детальний опис процесу перетворення NFA (після видалення λ-переходів) у детермінований скінченний автомат (DFA): [X]. Детальний опис процесу мінімізації (ключовий етап) DFA:	lexica__2025 lexica_part_dfa__2025 lexica__2025 lexica_part_dfa__2025 lexica__2025 lexica_part_dfa__2025 lexica__2025 lexica_part_dfa__2025 lexica__2025 lexica_part_dfa__2025 lexica__2025 lexica_part_dfa__2025 (KN1/)sp2019_soft__12345/ addon__09_11_2024/1 (KN1/)sp2019_soft__12345/ addon__09_11_2024/2 (KN1/)sp2019_soft__12345/ addon__09_11_2024/3
---	--	---

	[X]. Результуючий етап: Повністю детермінований скінченний автомат для токенизації (а: двовимірний масив для програмного моделювання; б: орієнтований граф з усіма переходами, окрім тих, що ведуть у <u>«мертвий» стан</u>, для графічного відображення): a[_], б[_]. Модель токенизатора на основі детермінованого скінченного автомату, оскільки це нетривіальне завдання теорії автоматів-акцепторів: [_]. Повністю детермінований скінченний автомат для розпізнавання підмови ключових слів у вигляді таблиці переходів (а: двовимірний масив для програмного моделювання; б: орієнтований граф з усіма переходами, окрім тих, що ведуть у <u>«мертвий» стан</u>, для графічного відображення): a[_], б[_]. Повністю детермінований скінченний автомат для розпізнавання підмови ідентифікаторів у вигляді таблиці переходів (а: двовимірний масив для програмного моделювання; б: орієнтований граф з усіма переходами, окрім тих, що ведуть у <u>«мертвий» стан</u>, для графічного відображення): a[_], б[_]. Повністю детермінований скінченний автомат для розпізнавання підмови беззнакових літералів із фіксованою комою (а: двовимірний масив для програмного моделювання; б: орієнтований граф з усіма переходами, окрім тих, що ведуть у <u>«мертвий» стан</u>, для графічного відображення):	(KN1/)sp2019_soft__12345/addon__09_11_2024/4 dfa_generator__2025 (проаналізувати програмну модель у файлі <i>matcher_by_dfa.hpp</i>) bult_src dfa_generator__2025 dfa_generator__2025
--	---	---

	a[_], б[_].	dfa_generator__2025
3.4. Розробка синтаксичного та семантичного аналізатора 3.4.1. Розробка дерев граматичного розбору 3.4.2. Розробка блок-схеми алгоритму 3.4.3. Опис програми реалізації синтаксичного та семантичного аналізатора	<i>Початковий етап посилається на попередній розділ (2.1), у якому формувалися EBNF та еквівалентний (пострічково) код для його перевірки.</i> Безпосередній етап: Синтез граматики відповідно до EBNF: [_]. Синтез множин FIRST та FOLLOW для отриманої граматики: [X]. Специфікація lookahead у правилах виведення граматики для моделі LL(2)-аналізатора: [_]. Форммування параметрів моделі LL(2)-аналізатора: [_]. Результуючий етап: Формування моделі LL(2)-аналізатора у вигляді недетермінованого автомата з магазинною пам'яттю (NPDA) (а: формальний опис; б: програмна модель): a[_], б[_]. Форммування моделі LL(2)-аналізатора у вигляді детермінованого автомату з магазинною пам'яттю (DPDA): [X].	EBNFVerify+GrammarVerify EBNFVerify+GrammarVerify EBNFVerify+GrammarVerify EBNFVerify+GrammarVerify dpda1_for_ll2_generator__2025 dpda1_for_ll2_generator__2025

3.5. Розробка генератора коду 3.5.1. Розробка блок-схеми алгоритму 3.5.2. Опис програми реалізації генератора коду	У цьому розділі виконується опис: алгоритму формування польського інверсного запису [_], алгоритму виконання польського інверсного запису з використанням стеку [_], алгоритму виконання польського інверсного запису з використанням стеку [_]. <i>Сам код генерації універсальний.</i> <i>Додатково можна описати процес формування SSA та інші аспекти.</i>	
4. НАЛАГОДЖЕННЯ ТА ТЕСТУВАННЯ РОЗРОБЛЕНОГО ТРАНСЛЯТОРА 4.1. Опис інтерфейсу та інструкції користувачу 4.2. Виявлення лексичних і синтаксичних помилок 4.3. Перевірка роботи транслятора за допомогою тестових задач	Описати консольний інтерфейс компілятора [_] Три тестові програми згідно варіанту [_] Коректна компіляція трьох тестових програм* [_] <i>*у записці наводиться тестування лише трьох заданих тестових програм, але під час захисту потрібно вміти написати вашою мовою програмування будь-яку просту тестову програму</i>	cw_sp2__2025_2026/ src/implementation/cli base_test_programs_2025 cw_sp2__2025_2026 (запуск)
ВИСНОВКИ СПИСОК ЛІТЕРАТУРНИХ ДЖЕРЕЛ ДОДАТКИ А. Таблиці лексем для тестових прикладів Б. С код (або код на асемблері), отриманий на виході транслятора для тестових прикладів; Б. Документований текст програмних модулів (лістинги)	Оскільки це курсовий проект (а не курсова робота), то окремо у форматі А3 (що має бути оформлено як стандартне креслення) подається один графічний матеріал. Це може бути приміром загальна блок-схема роботи транслятора: [_]	